

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

A state of the art analysis of GPU computing with Rust

Tesi di Laurea Magistrale in
Computer Science and Engineering - Ingegneria Informatica

Author: **Daniele Locatelli**

Advisor: Prof. Alessandro Margara
Co-advisors: Gianpaolo Cugola
Academic Year: 2022-23

Abstract

In recent years, we witness a steady increase in the availability of data, thus, in solutions that enable its analysis. At the same time, hardware accelerators like GPUs are becoming more and more accessible.
Therefore, in this thesis we will consider Noir, a framework written with the Rust language, and analyze possible solutions that allow it to perform tasks by exploiting the computational capabilities of GPUs.

Our work begins by researching existing Rust libraries that allow to perform computations on the GPU.
For each found library, we will analyze both the intrinsic features, and the functionality they offer compared to the most known and most widely used libraries, such as CUDA and OpenCL.
Finally, some will be chosen, studied and compared in more detail, to fully understand their potential and performances.

Keywords: Rust, GPU, Streaming Data Analysis

Abstract in lingua italiana

In questi anni si sta assistendo ad un costante aumento di disponibilità dei dati, e quindi di soluzioni che ne permettano l'analisi. Allo stesso tempo, acceleratori hardware come le GPU diventano sempre più accessibili.
Perciò, in questa tesi prenderemo in considerazione Noir, un framework scritto con il linguaggio Rust, e analizzeremo le possibili soluzioni che gli permettano di eseguire le operazioni sfruttando le capacità di calcolo delle GPU.

Il lavoro comincia con la ricerca delle librerie Rust già esistenti che permettano di eseguire calcoli sulla GPU.
Di queste verranno analizzate sia le caratteristiche intrinseche, sia le funzionalità che offrono rispetto alle librerie più conosciute ed utilizzate, come CUDA e OpenCL.
Infine se ne sceglieranno alcune, che verranno analizzate e confrontate più nel dettaglio, per comprenderne a pieno le potenzialità e prestazioni.

Parole chiave: Rust, GPU, Analisi di streaming di dati

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

Introduction **1**
0.1 Context . . . 1
0.2 Goals . . . 1
0.3 Problems to solve . . . 2

1 Background **3**
1.1 APIs . . . 4
1.2 Graphics API . . . 6
1.3 Intermediate representations . . . 8
1.4 Noir . . . 9
1.5 Rust . . . 10

2 Found crates **13**

3 Analyzed solutions **17**
3.0.1 Single pipeline . . . 17
3.0.2 Multiple pipelines . . . 20

4 Figures, Tables and Algorithms **27**
4.1 Algorithms . . . 27

5 Conclusions and future developments **35**

Bibliography 37

Introduction

0.1. Context

Something that characterizes modern days is the huge amount of data that is daily generated in every aspect of our lives, and this number increases every year. It has been estimated that in 2023 we will create data for 120 zettabytes, which corresponds to around 328 million terabytes per day, and predictions show that this trend is not going to stop. Even if half of it is composed of video contents, it is still an incredibly big amount of data[6]. But data by itself is nothing. This is why it is necessary to have technologies that are capable of analyzing it and provide useful insight into the information it carries; and as the volume of available data increases, then the speed at which we analyze it must do so as well, so that we are always able to consider the biggest amount possible. But speed is not the only goal to achieve. In fact, another crucial aspect about current data is that it becomes also more complex, so in addition to speed, it is important to have solutions that allow to possibly define all kind of operations on data.
This is why Noir [5] was developed. It is a framework for streaming data analysis that has been proven to be faster than its competitors, while maintaining an high level interface and wide expressiveness on the operations that can be performed on data. In fact it also allows to define custom ones, while providing some already predefined.

0.2. Goals

After reaching these good results, it is time to set another goal and further improve the framework capabilities. In fact, we chose to aim at a higher speed by integrating GPUs into Noir, as they are a very good tool to use when handling with a big amount of data, something that Noir is specifically designed for. If this would be achieved, then the framework will not operate on data serially, using the CPU, but it would somehow pass data to the GPU, along with the operation(s) to perform on it, and then Noir would retrieve the results from the GPU when it finished its job.
We have some wishes regarding the technical features of any solution we might find. The

most desirable solution is the one that allows us to program the GPU using Rust, so that we will only have to know a single programming language, furthermore we would like that the code can be easily passed to the GPU along with the data that it has to analyze. This desired scenario is broadly analyzed in this paper [10]. More in detail, we are looking for a library that resembles the operations that must and can be performed using the CUDA toolkit, a very fast programming interface that runs only on Nvidia GPUs ([11]).

0.3. Problems to solve

To achieve this goal in its entirety, there are many aspects that we must take care of. These are: how to write the code that the GPU has to run, how to pass it to the GPU, how to transfer data between the processing units and how to synchronize them.

1 | Background

A GPU is a hardware accelerator that allows parallel computation on a big amount of data. This is possible because GPU cores have a simpler architecture compared to CPU one.

Core concepts. A GPU core runs a single thread and is specialized at data processing, opposed to CPU cores that also take care of data caching and managing the control flow. Of course these aspects must be taken care of somehow. In fact, there is a control unit common to multiple cores that deals with the latter issue, while the memory must be handled by the programmer. This makes it necessary to know in greater detail the memory hierarchy.
There are three levels of memory, starting from the biggest and slowest we have: global memory, shared by all the cores; shared memory, accessible only to a subgroup of cores; private memory, reserved to a specific core. The just mentioned subgroup of cores is called *block*, and they are organized into grids. Threads running in a block can cooperate using the thread synchronization primitive, which is a function that stops the execution of the thread calling it until all the others in the block reach it. This helps to be sure that some specific operations is performed by all threads. Since GPUs follow the SIMD paradigm, which means that threads execute a Single Instruction on Multiple Data, the synchronization might look useless. But this is not the case since there is another lower level of thread grouping, called warp, obtained by splitting once more a thread block. Different warps composing a block can be executed at different time, and here is where the synchronization is needed. In general, all the threads running in a single warp must execute either execute the same function of the other threads or none (this is where the GPU is SIMD). In fact, if there is a branch in the kernel and different threads belonging to the same warp follow different paths, then the execution will be serialized, which means that all threads following a certain branch will run, while others are idled until the running warp finished its operation, and another warp will start.

GPU operations. To interact with the GPU we must write a kernel or a shader. The name changes according to what environment we choose, but the concept is the same: it

is a piece of code that will be run by the GPU on the data available in global memory. Since the GPU is an external device, it is necessary to load data on it before starting the computation, and to read data after the computation is completed. To do this, the PCI express bus is used, which is slower than the bus used by the CPU for normal data transfer. This speed loss can be overcome by overlapping computation and data loading/reading. In general, handling data is a delicate matter when programming GPU because of the SIMD paradigm: GPUs do not control any data race, so if the kernel (or shader) that has to run is badly written, then a single data can be accessed and modified by multiple threads, without knowing who accessed to the correct data, and who to the modified one. It is the programmer must specify in the kernel what data must be accessed by a thread, relying on the fact that all of them are assigned some IDs that represent their position in the block, and the position of the block containing it. In this way, we can address the right data in global memory using the thread and block index. Since blocks are organized in a 3D fashion and blocks are composed of a 3D grid of threads, their indexes are represented by a 3D variable. Finally,blocks are grouped into

1.1. APIs

Now we focus on the interfaces available to exploit GPU capabilities, namely GPU APIs, and the programming languages that these require, since they are closely tied. When dealing with these kind of API, the terms *host* and *device* are often used, where *host* is the machine from which the GPU is handled, while *device* is the GPU itself.
We can define two categories: general purpose (GP) and graphics API. As the names say, general purpose ones do not target a specific task, while graphics APIs mainly focus on the drawing and rendering capabilities of GPU, even if they still allow programmers to use the GPU for computations.

General purpose API

Two common GP APIs are CUDA and OpenCL. Between the two, OpenCL is a higher level API because it can run on different devices and different architectures, while CUDA can only run on Nvidia GPUs. Because of this, CUDA does not have a device initialization phase, while OpenCL does. Passing data between CPU and GPU differs as well. CUDA provides a function to perform memory allocation directly on the GPU, where later data will be copied to; OpenCL allocates on the host a buffer containing data, then memory will be allocated on GPU and data will be copied on the just allocated space, in one only operation visible to the programmer. Another consequence of the possibility of having

different targets is that CUDA kernels are compiled at compile-time, while OpenCL one are compiled at run-time, again because the running device is in general not know until the code runs. When we want to run the kernel, it is important to specify how many blocks we want to run and how many threads compose each block. Kernels and the CPU code are written in both libraries using an enriched version of C, but each API uses a different language.
We now report some differences in the names of built-in variables to the kernel. (DOMANDA: serve? Può essere utile per quando si legge il codice dopo)

CUDA	OpenCL	CUDA	OpenCL
gridDim	num_workgroups	block	workgroup
blockIdx	groupId	blockDim	localSize
threadIdx	localIdx	blockIdx * blockDim + threadIdx	globalId

Table 1.1: Different names of built-in kernel variables

GPU operations. In order to issue instructions on GPU, such as running kernels and loading data, CUDA requires rather simple function calls that, in basic situations, are executed immediately after, while OpenCL needs a more complex structure, and the execution will be performed only at the end, when everything is set. In fact, we must define a variable called *CommandQueue*, where we store all the operations that the GPU must perform. Each *CommandQueue* refers to a single device, but multiple *CommandQueue* can issue instructions to the same GPU: this can be used if the instructions belonging to the different *CommandQueue*s do not have to be synchronized. Eventually, this will be passed to the GPU that will take care of running the predefined operations. This last case, with multiple *CommandQueue*s, is helpful when we want to overlap operations, for instance data loading and computing. CUDA, instead, defined a type called *stream*, that enqueues asynchronous operation. So, it is enough to define multiple streams to which we assign different operations, then each stream can run operations in parallel with the others.

Synchronization. Since there are multiple parallelism layers, both on GPU using kernels and on GPU commands, some synchronization primitives are required. There are multiple possibilities.
Starting from the threads running on GPU cores, we might need that all threads in the block have reached a certain kernel instruction: to solve this OpenCL and CUDA provide execution barriers.

Both allow to synchronize memory operations as well, thanks to memory barriers, called fences by CUDA. When it is called on a specific memory, either shared or global, it means that we want all the other threads to see the changes we have done on it. When using CUDA, execution and memory barriers are performed with the same instruction, while OpenCL allows us to choose. Furthermore, OpenCL allows us also to choose on which memory, global or shared, we want to place the barrier.
The next synchronization feature is called semaphore, and it is used to define a dependency between GPU operations, needed for APIs interoperability, which means that different APIs can interact with each other. This of course will require some synchronization between them, and this is where semaphores come in handy: they allow to define an dependencies between different tasks issued on the GPU.

External memory. This is not the only way CUDA and OpenCL can use to cooperate with other APIs, in fact they can also use memory allocated by other APIs to directly operate on it, without the need to copy data. This mechanism is called external memory. Both CUDA and OpenCL give these interoperability possibilities, by relying on other APIs, for instance Vulkan, which we will talk about later.

1.2. Graphics API

Core concepts. The structure of these API is completely different compared to GP APIs, since, as anticipated, these focus on rendering, so they have a structure that allows to perform the multiple steps required by it. All these steps are called *stages* and here we give some examples: vertex stage, which places each vertex composing the figure to represent in the right position in space; fragment stage, which assigns the proper color to the vertices. In order to instruct the stages on the operations that it has to perform, we must pass a *shader*, which conveys a concept that is similar to the previously defined kernels. The difference is that since each stage has to perform operations on a single element, it is not required to perform synchronizations, to use shared memory, and to say how many *shaders* must be run. Shaders here are grouped into workgroups, that share the same characteristics of the GP APIs. All the stages, after they are fully defined, are combined into another object called *pipeline* that represents them all, and contains the variables that must be elaborated by all stages. These variables must be loaded on the pipeline using buffers, an object that can contain more than one, like a struct. We must define its structure both on the host and device. On the device it is easy: we simply need to define a struct with all its contained variables. On the host, it is necessary to

use a type called *DescriptorSetLayout* (DSL) that describes how buffers are composed. Now, every time we want to create a buffer, we need to indicate its layout (or structure) using a DSL and assign to each element in the DSL the actual variable; this will give us an object called *DescriptorSet*, an implementation of the generic DSL. Finally, these must be related with the structs defined in the shader, through a procedure called data binding. Each correspondence between the host and device variable is described by a tuple of values: the descriptor set index, that is an index over the loaded buffers, and a binding index, that the programmer changes according to how often that buffer is used (the higher the more changes). This helps to allow the pipeline to know that some buffers are bound once and then rarely modified. Of course, this increases the performance.

GPU operations. The general structure of the code needed to run used by these APIs is not very different from the one needed by GP APIs: we need to setup the environment, load data on GPU, execute the shader and finally retrieve the results. All of them, though, have at least some difference in either the names of the concepts, or the data structures used.
In fact, the initialization takes care of defining all the structures needed later, like DSL and Descriptor Sets, which are used in order to perform data loading.
As previously said, the computation phase does not require specifying the amount of shaders that must be run, but this is not true if we use the API for computing purposes. In that case we must choose how many shaders to dispatch, and the size of the workgroups, like in GP APIs.
Finally, when the GPU has completed its operations, results can be retrieved using the primitives that allow to read the GPU allocated buffer, instead of copying between memory at a lower level.

Synchronization and external memory. It is possible to synchronize these operations in both Vulkan and Wgpu: they support semaphores, fences and barriers. This was expected for Vulkan, since we have said that OpenCL relies on it for semaphores, but it was not granted for Wgpu. Wgpu, for instance, does not support external memory yet, even if there are some attempts in that direction.

Differences. The first thing we notice is that they target a different abstraction level. In fact, Vulkan is an API that only requires that drivers for the running OS and GPU were developed, then the programmer can exploit all the available functions to operate in great detail on the GPU.
Wgpu, instead, focuses on portability, in fact it runs on multiple back-ends, Vulkan or

Metal for instance. Another consequence of the fact that Wgpu is a higher level, is that it allows the user to write shaders using a programmer friendly language, WGSL or GLSL, and it takes care by itself of the compilation into the actual language needed by the underlying back-ends. Vulkan, instead, expects a SPIR-V file, on which we will talk more about later. There is no big difference between WGSL and GLSL, they almost provide the same functionalities, but the former was created to be easily supported by web browsers, and this is why Wgpu uses WGSL: this API was specifically developed to support web applications as well.
These two shading languages though, have some differences with the previously introduced CUDA. In fact, CUDA, using warp votes functions, allows to make running warps evaluate a predicate; allows to define functions for structs, allows to use recursive functions, while WGSL and GLSL do not have these features. On the other hand, since these two languages are used for shader operations, provide some algebraic built-in types to represent vectors and matrices (vec2, vec3, mat3, mat4, for example), along with a variety of built-in vectorial and matricial transformations, like the dot product, the cross product, the inverse and more. Then WGSL and GLSL rely on buffers to retrieve the variables that they need, as previously explained, while CUDA declares them as simple variables, only adding an additional keyword that specifies where that variable resides.

Another difference between Vulkan and Wgpu is in their host code language: Vulkan needs to be programmed using C, while Wgpu relies on Rust.

1.3. Intermediate representations

It is common that shaders are translated into an intermediate language, instead of the language required by the specific GPU. This helps to keep the compiled code more generic, in order to allow targeting multiple drivers. In fact, it will be the specific driver that will translate the intermediate representation into a the right code, specific to the available GPU.
The intermediate language we will use the most is **SPIR-V**; for instance for Vulkan. It was developed by the same group that developed Vulkan, and all the shading languages that can be translated into it, for example GLSL and WGSL, after the conversion into SPIR-V can be automatically used by Vulkan drivers.
For what concerns CUDA, it relies on **PTX** and **NVVM**, two intermediate languages that were developed for Nvidia GPUs. The main difference between the two is that PTX, since it is architecture specific, represents lower level instructions compared to NVVM.
Finally, we introduce **Fatbin**, which is another low level representation of CUDA shaders.

This differs from the others because it contains multiple versions of compiled shaders, in order to be used my multiple architectures, hence it contains multiple compiled PTX files.

1.4. Noir

Starting point. The dataflow model is an abstraction that focuses on the operations to be performed on the data, not on their implementation. It defines a start node that feeds data to the model, an end node that gathers results and as many intermediate nodes as the number of operations to be performed on data; these nodes are linked by the necessary arcs to represent their dependency. This model does not consider implementation details and allows great flexibility: each computation can be executed on different machines, several machines can share the work of a single computation or several computations can be executed on the same machine. This model also provides functionalities to define batches and windows. These concepts are useful because in data-streaming analysis, data is not considerated immediately, rather data is grouped and considered only when some conditions are met. So we define a batch as grouped data that is ready to be sent to the next block for further analysis; windows as operators that instruct the next one on how to group data to perform its analysis. For example, the Max operator will be instructed by a window on which values it should compute the maximum.

Structure. Noir has been implemented on this model. This new framework aims to offer great expressiveness by relying on the dataflow model while achieving high analysis speeds on big data. Contrary to the dataflow model, in Noir it is necessary that each operator is instructed on how to modify the data it receives by passing a closure. (QUESTION: there is no mention in the dataflow paper of anything like closure, so I assumed that it does not require it) Once this is done, the model is fully defined, and two types of operators can be distinguished: stateful, which use internal variables in order to complete the transformation (Average operator); stateless, which operate on one data item at a time (Doubling operator). Arcs, on the other hand, can be seen as a representation of the *stream* object defined by Noir.
Each operator can have multiple input and output streams, and when an operator leads to an organization of the data in the output streams that is different from that in the input streams, there begins a new partitioning of the operators into a new block. For instance, this happens if you run the GroupBy operator, which will partition the input into different streams depending on the value of the data. This distinction is useful when considering opportunities to execute operations in parallel; in fact, if you have a partition with multiple streams, each stream can be considered independent of the others, and

therefore executable independently. Also, it is useful to know how blocks do not receive data individually, but in batches: every time a block is defined, an additional operator is placed as last, which acts as a buffer that holds the data until certain conditions are met, such as having reached a certain number of elements, or having waited for a predefined time interval. This is exactly the data that is used by the operators. Since we plan to introduce GPU computation, these batches will be the inputs that the GPU will have in order to perform the computations.

Noir and GPU combination. Now we need investigate what can be done to combine Noir and GPU, namely, how to handle operators and data, if there is a possibility that the machine we are operating on is equipped with a GPU. To do this we need to start from blocks. So it is necessary to understand whether it is possible to execute a block entirely on the GPU, which means to execute serially the operations defined by its component operators, and to execute the internal flows in parallel. In addition, it is necessary to understand how we can behave if two machines, one equipped with a GPU and one without, individually run two independent nodes; in practice, we need to understand how to synchronize operations. Finally, we want to investigate the various options for telling the GPU which operator should be executed, and with which closure, and then extended to the case with multiple operators.

FlatMap operator To explain how this goal can be achieved, for each proposed solution we will try to give an idea of how to implement a Noir operator called FlatMap, a richer version of Map. In fact, the latter considers one element at a time, performs some operations and eventually returns one result for each input value. Instead, the first one takes as input a single element, performs the same operation as if it was a Map operator, but then returns as result a predefined number of elements. In both cases, the collection of outputs returned for each input element will be combined into a vector.
For instance: if we consider a square operation on the array [2 -4 1], Map will return [4 16 1], while a possible result of FlatMap can be [4 4 16 16 1 1], where we can see that for each input, it returned twice the result.

1.5. Rust

As previously said, Rust is the programming language used by Noir. It is famous for its safety features and performances, where one thing does not sacrifice the other. In fact, Rust performs many checks at compile time, like that variable lifetimes and variable ownership is respected, preventing memory leaks. It provides abstraction at zero cost: for

instance, iterators and closures, some high level constructs, are efficiently translated by the compiler in a way that does not require many expensive runtime checks.
As we said, these improvements come with the cost of stricter rules that must be followed when writing the code. Rust also allows to avoid respecting those rules marking the code with the $unsafe$ keyword. Of course, this practice must be avoided the most possible.
Eventually, some terminology. Rust has defined the term *crate*, in order to define a runnable code. But this can be either a binary or a library, so it can be runnable code or code used in other runnables. When we refer to *crates*, we always mean library crates, unless differently specified.

2 | Found crates

We now present a comparison of the crates we have found and how they respect our requirements. The criteria are still the ones we previously used to compare the previous APIs, but now we group them in a table, in order to be more clear. We will provide further details after the table (on asterisks as well). (ORA QUESTA TABELLA È UNA FIGURA, HO PROVATO CON LATEX MA ESCE DAL FOGLIO. PER LA VERSIONE FINALE DELLA TESI CERCO DI FARLA COME TABELLA)

CRATES	Target	Supported synchronization primitives	Supports parallel operations	Supports external memory	Shader/kernel language
		Graphic APIs			
Ash	Vulkan API	Fences, semaphores and barriers	Yes	Yes	SPIR-V
Vulkano	Vulkan API	Fences semaphores, and barriers	Yes *	Yes	GLSL
Rust-GPU	Ash or wgpu	Fences, semaphores and barriers	Yes	Yes, using Ash as back-end. No, using WGPU	Rust
		CUDA			
Rustacuda	CUDA drivers	None	No	No	PTX
Rust-CUDA	CUDA drivers	Fences and barriers	Yes *	No	Rust
		Other kinds			
Rust-gpu-tools	Rustacuda for CUDA and OpenCL3 for OpenCL	None	No	No	Fatbin, when using CUDA; string, when using OpenCL
Emu	Wgpu	Fences semaphores, and barriers	Yes	No	GLSL
Arrayfire	Its own SDK, runs on any OS. Uses unified back-end	The API has no such concept	The API has no such concept	The API has no such concept	The API has no such concept

Figure 2.1: Caption of the Figure to appear in the List of Figures.

Following the order of the crates in the table, we will comment the results.

Ash This crate [3] provides a low level Rust binding to the Vulkan API, in fact nothing regarding Vulkan is hidden to the programmer, who must take care of all the aspects of the Vulkan API, for instance all its structures. Of course, this helps when we need to use

the crate to solve different problems, but this is not our case. In a way, it also moves in a different direction with respect to Noir, as one of the goals of Noir is to provide a high level interface to the running code, something that this crate does not do.
Another problem with this library is that, as stated in the Github repository everything is unsafe, so it also breaks one of the main constraints that make Rust is a good programming language.
Finally, it requires that the shader code is compiled; this is not a great problem, but it adds a step to the workflow that we would like to have.

Vulkano This crate [15] too provides a Rust binding to the Vulkan API. but it lays at a slightly higher level compared to Ash, for example, by providing some types that represent the low level Vulkan types and making them easier to manage. Unfortunately, also this crate does not fully match our requirements, since it does not provide a way to pass a closure to the GPU. On this matter, it is preferable to Ash, since it handles by itself the compilation to SPIR-V, so we can write shaders using GLSL. Furthermore, it takes care of using safe code the most possible.
The asterisk refers to the fact that in theory, as it relies on Vulkan, this crate should be able to run multiple pipelines in parallel, but our tests suggest something different, as shown here 3.1.

Rust-GPU This crate [7] focuses on wide applicability, in fact it allows to choose between running the code through Ash or Wgpu (remember that Wgpu itself runs on multiple back-ends). It is interesting because shaders here can be written in Rust.

Rustacuda As can be seen, this crate [14] represents a basic solution to running CUDA kernels with Rust: it does not provide anything more than that and it only requires a compiled *.ptx*. But because of its simplicity, it cannot be considered a solid starting point for a problem like ours.

Rust-CUDA This crate [13] is very interesting since it allows to write kernels using Rust, and it targets the CUDA API. The asterisk refers to the fact that it deals with CUDA streams for asynchronous operations, but only when it comes to running kernels, which is good, but it would be more interesting if it also handled data transfer. Unfortunately, the last update of the crate was in July 2022 and it has not been kept up to date with the changes in the used dependencies since then, so it is not possible to run the code available on the repository. After many tries and intermediate bug fixing, it still did not run, so we chose to not take it into consideration any longer.

Rust-gpu-tools This crate [8] is interesting because it allows to write shader code both for CUDA and OpenCL, so we would be able to target an efficient API (CUDA) or a more generic API (OpenCL) if the first is not available. It is worth noting that it relies on Rustacuda when we choose to run using the CUDA API, and on OpenCL3 crate ([4]), when we want to use OpenCL. In general, it can be seen from the functionalities provided that it still is at a very early stage, so we excluded it from any further consideration.

Emu This crate [16] runs on Wgpu, but tries to make the code structure look like a CUDA program. The additional advantage of Emu is that it uses GLSL shaders, instead of the less stable WGSL. Since this crate does not provide any advantage with respect to the others, we did not analyze it in greater detail.

Arrayfire This crate [1] works completely differently from the others, since it tries to hide the most possible the executor of the operations. The owners provide SDKs for the most common OSs (Windows, MacOs and Linux) and, thanks to a back-end they defined (Unified Backend [2]), the crate can run on CUDA, OpenCL and on the CPU, as well as change the back-end. Because of this possibility, it abstracts completely from the hardware, so no extensive primitives to handle the GPU are available: it provides functions to operate on vectors and they deal with the GPU aspects. If needed it still gives the possibility to handle the GPU data transfer using CUDA or OpenCL. This crate provides a big number of operations that can be performed on data, but unfortunately it is not possible to define custom operations, so we had to exclude it from the possible solutions.

3 | Analyzed solutions

Because of the just said motivations, we analyzed in greater detail only Vulkano and Rust-GPU, out of all the crates we have found. We now show some similarities and differences between the two, while giving a hint on how to write the code required to perform the previously explained FlatMap operator (1.4). We then show the performances of the analyzed crates, obtained implementing a program to simulate the SumReduce operator, which gives the sum of all the elements that compose the input.
We will not show all the implementation details since they can be retrieved from each crate's repository, where a full example of how to run a program that uses the compute shader is available.

3.0.1. Single pipeline

Writing the shaders

The first step consists of writing the shaders, as it is needed when we will create the pipeline object.
In practice, writing the actual code can be done at any time, since the host code only requires the path of the file that contains it, but it is logical to start with this step.
One of the goals we want to reach to solve our problem, is to have a wide expressivity of operations that can be performed on the data we want to analyze, and using shaders, we can say that this goal is easily achieved: we have shown before that the languages used for shaders are expressive.

Similarity: shader structure. The most important thing that they have in common is that both of them must be programmed using the graphics pipeline structure that was explained earlier (1.2). Remember that here we are interested only in the Compute stage, since no drawing needs to be performed. It is worth noticing that each pipeline has a single Compute stage, so if we want to execute multiple compute shaders, we will need to

pass each one of them to a different pipeline. But the mechanism is always the same.
Before explaining the differences between the two crates on how they handle shaders, we explain more in detail how they must be composed.

Shader composition. As we said, when writing a shader it is important to define its inputs and how many times it will be run in each single workgroup (group of threads that share the same memory). This latter element is composed of a predefined number of threads that run in parallel and share some memory for very fast exchange of information. It is the programmer that chooses the number of threads that compose a workgroup. Since we want to reach high speed, it is crucial to choose the right amount of threads share the same memory: it is not advisable to set this number too high, in order to allow the GPU to easily perform the proper scheduling of these workgroups, and not too low, to allow a certain freedom in exchanging information between threads.
To not lose focus on the shaders, we will explain how to define its inputs later.
After defining how shaders are grouped and the data needed they need, we can write the function that operates on the input. A reason why we have chosen this crates it that both of them perform some checking on the code at compile time, when both of them transform the written shader into SPIR-V; the ones using OpenCL do not perform any compile time checking.

Difference: shaders language and compilation. The differences between crates on this matter lie in how shaders are defined and handled. In fact, Vulkano requires that the code is written into GLSL, and it handles the compilation thanks to the combination of two modules: a procedural macro called $vulkano - shaders$ that calls the compilation of the shader and then handles the results, and the Shaderc-rs crate [9] that takes care of the actual compilation from GLSL to SPIR-V.
Rust-GPU instead expects a shader written using Rust, with the addition of some keywords, that will be later transformed into instructions for the SPIR-V code generation. Finally, Rust-GPU compiles the shader using $rustc_codegen_spirv$, a crate they created that can be found in their repository.

Pros and cons. Not using Rust as a shading language is a possible minus as it can be a plus, in fact writing shaders using a language that was specifically designed for that is possibly desirable. In our case we can see that using GLSL provides more functionalities compared to using Rust, which has to be converted later. As it is possible to see from Listing 4.3 and 4.4, that show the shader for each crate, GLSL provides a vari-

able representing the workgroup size, while the Rust-GPU suite does not. In the latter case then, it is convenient to define one ourselves, in order to keep a similar code structure. This flaw can be seen once more by looking at the crate code, specifically, in file $rust-gpu/crates/rustc_codegen_spirv/src/symbols.rs : 64$, where we can see that the specific keyword is not handled, since it is constant. This is not a real problem because we are the ones defining how big a workspace is, but it shows a flaw of relying on an intermediate crate and representation.

initialize the variables

After defining the shader, we initialize the variables required to create a pipeline (like the Descriptor Set Layout), as shown in repository of each crate.

Data allocation

Now we show how to define the shader inputs and how we must choose the required space that must be allocated for data.
In principle, if n is the number of times that the result is repeated in the output, we would need space to store n times the length of the input array. In this way, the GPU has the right amount of space where to store the repeated output to. But the allocation step must be done properly thinking about the constraints imposed by parallel computing, since the number of arrays to be instantiated on the GPU can vary depending on what kind of results we need and the properties they have to satisfy.
We can identify two cases:

- if the repeated outputs must be right next to the original one, then we need two vectors on the GPU (input and output). In fact, element in position x of the input ($input[x]$) will have to be modified and the result stored in $output[x]$ and $output[x+1]$. If we used a single array in this situation, then $input[x]$ will be stored in $input[x]$ and $input[x+1]$, but $input[x+1]$ will contain a value to be analyzed that will consequently be lost
- If the repeated outputs can be in a distant position from the original values, then we can simply allocate on GPU a vector with length $n * original_length$ and each element will be mapped from $array[x]$ to $array[x]$ and $array[x+original_length]$. This can simply be done in parallel without any risk of data deletion.

For both crates we address the first problem, since it is a more complete case.

Actual memory allocation. As previously stated, we must allocate two vectors on the GPU, one that stores the input, one for the results. Allocating the vector containing the input can be done as in Listing 4.1 and 4.2, for Vulkano and Rust-GPU respectively. Something similar can be done for the output, again as in Listing 4.5 and 4.6, for Vulkano and Rust-GPU respectively. The shown algorithms also populate the buffer, so at this point we have both allocated and loaded the data.
(DOMANDA: ho tolto che $.from_iter()$) non va, giusto cosi? Spiegavo che ci sono due funzioni per caricare dati, ma from_iter non funziona)

Allocation limit. The two crates have different limits when it comes to the amount of data that can be allocated on memory. In fact, the only limit imposed by Vulkano is to not exceed the amount of space available on the GPU, which is normal. Instead, Rust-GPU sets a limit to the amount of data assignable when allocating a buffer, that can be seen in $wgpu - types/src/lib.rs : 943$, and is lower than the highest allocable amount. As we will see in the following section, this limit is somehow valid for Vulkano as well.

Data binding

Once buffers are allocated and data is loaded, it is necessary to bind the buffers on the GPU to the correct variable defined in the shaders. The examples show exhaustively how this can be done.

3.0.2. Multiple pipelines

What said so far is useful if we want to take care of a single operator and a single stream. But, since it is also interesting to see how to run pipelines both in series (multiple operators) and in parallel (multiple streams), we report how this can be done, and what we have learned about the two crates on this matter. We performed our tests with an implementation of the SumReduce operator, instead of the FlatMap, but changing the implemented operator does not influence the results.

Difference: pipeline handling. It is good to remember that to run a compute shader we need a compute stage, and for each compute stage we need one pipeline. So if we want to run different shaders, then we need different pipelines.
Rust-GPU easily takes care of this. If we want to run two pipelines in parallel it is enough to create two pipelines and assign to them the separate informations they require, as if they were on two different machines, then submit two execution commands to a

single executor. This last step is shown in Listing 4.8.
Running two pipelines in series is easier, and it is enough to request to the executor that the execution of pipelines must be done following a specified order (Listing 4.7).
Vulkano instead is not as flexible. It allows to run pipelines in series, running code as in Listing 4.9, but not in parallel. We arrived to this conclusion performing many tests and trials, we always noticed that the computing phase of the algorithm always took twice the time that was required when running a single pipeline, as show in Table 3.1.

Performances

Benchmark setting. To benchmark the speed of the crates, we used a combination of to crates, Criterion and Micrometer [12]. We used both because the first allows to benchmark the entire code, while the second allows to split the code into multiple sections. Then we performed the benchmarks varying some **parameters**: the input size and the workgroup size. Since the outcomes turned out to be very similar, we just report the case in which $workgroup_size = 128$.

Outcome: series. As we said already, to implement an operator we must write a shader. But the SumReduce on GPU must be performed in two steps, so we need two shaders: the first one performs a partial sum on the elements of the array, the second collects all the results. These two shaders must be executed sequentially, hence we need a series of pipelines.
Before looking at the outcomes in the different sections, we want to highlight an interesting behavior of Vulkano. It is not clear why this happens, but we can see that when the input size exceeds the input limit of Rust-GPU, then the performances of Vulkano sensibly increase.
We now give a brief overview of what each section measures, along with the results obtained and some comments.
(DOMANDA: sarebbe meglio spostare l'elenco fuori da questa section? L'ho lasciato qua perché almeno può contenere le figure, dato che per il caso con le pipeline parallele non riporto figure. Altrimenti ho provato a fare una pagina con solo figure (come da template Poli), ma ho visto in altre tesi che non viene mai usato)
These are the defined sections:

- Initialization: here we will define the variables that will handle the device (GPU). As expected, the time needed to run this phase does not change when changing the parameters.

- Pipeline setup: here we create the variable representing the needed pipelines, and we assign a shader to them. This section as well requires a rather constant time to be executed.
- Buffer allocation and data loading: here we define the space on GPU for each array that must be loaded and then load its values. Of course, this varies as we vary the length of the data to allocate. We notice an unexpected behavior though: Vulkano does not show a monotonically increasing time, as the data increases.

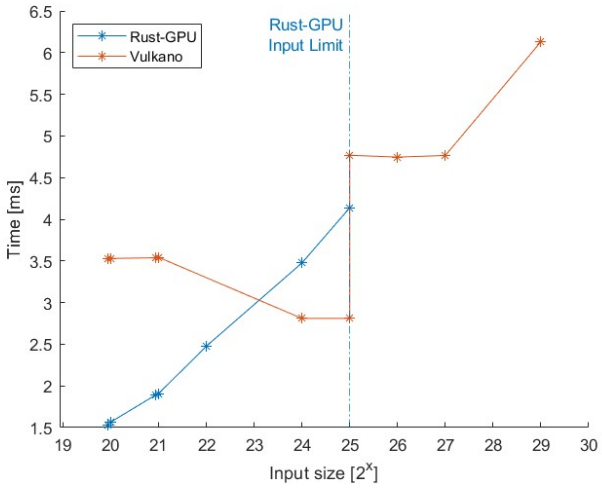


Figure 3.1: Data allocation and loading

- Data binding: here we simply bind the allocated buffers to the corresponding variable in the pipeline. The execution time is rather constant.
- Computing: in this phase the GPU performs the operations and both shaders are executed

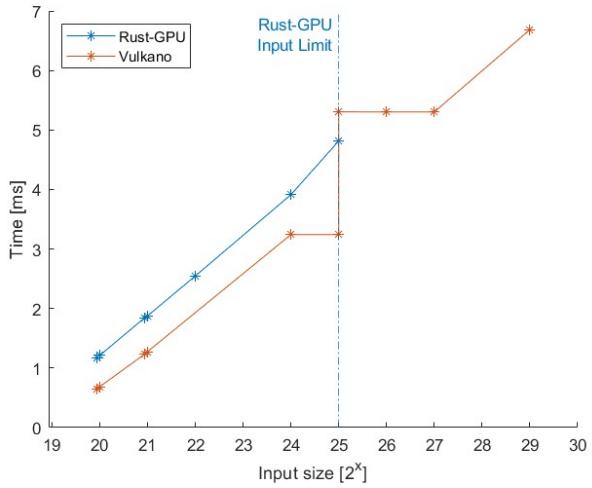


Figure 3.2: Computing

- Data reading, here we copy the results of the computation from the GPU back to the CPU, and also here we see that the time required increases as the amount of data to read increases, as expected, except for an unexpected spike.

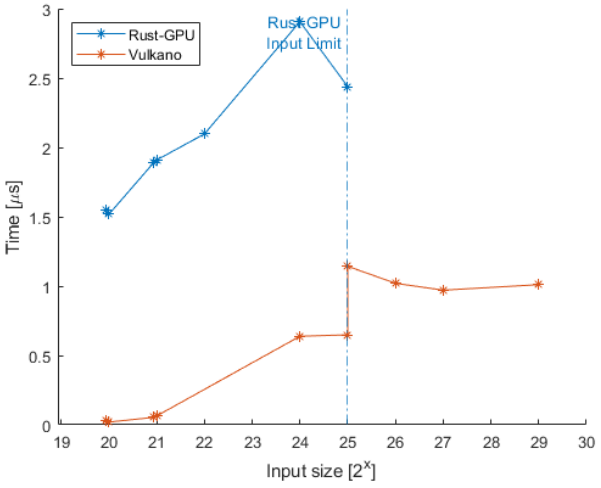


Figure 3.3: Data read from GPU

At last, we show the general improvement obtained using the GPU. The CPU here performs a for loop over the input elements, while the crates run all the sections just described, so if we focused on the compute time only, this difference would increase.

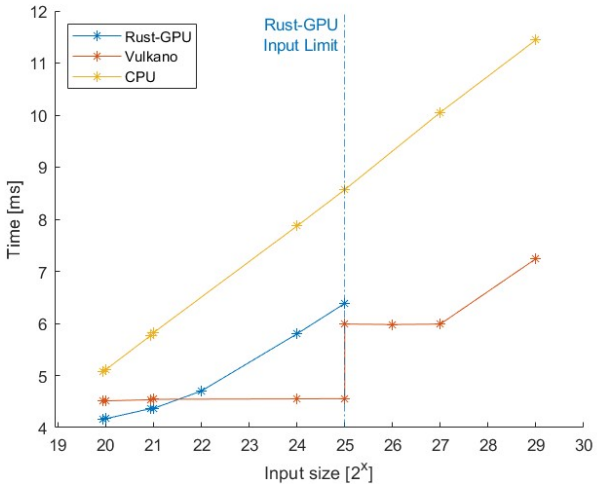


Figure 3.4: Crates and CPU only comparison

Outcomes: parallel. Given the limitation imposed by Vulkano, it is not possible to properly compare the crates in this case. We will just report one case for Vulkano that shows how the computing time doubles, suggesting that the pipelines are not actually run in parallel, and one case for Rust-GPU. Both tests are performed with an array of one million elements as input, which is rather small for GPU computing. This was intended: if we see this behavior with such a small amount of data, as the size of the array increases, the same can be seen even more evidently.

In order to perform these tests, we needed a different way of computing the SumReduce, since we could not use two different shaders in series. We implemented a shader that at each step would sum the values in one workgroup, and store the result on an output array in the position equal to the index of each specific workgroup. This procedure is performed until there is one only workgroup performing the sum, and its outcome would be the final result. Because of this way of computing, we increased the number of threads in a workgroup to 256, since we needed a bigger workgroup size to sum in parallel a good number of values.

These are the found results

N. Pipelines	Init	Setup	Data alloc/load	Data bind	Compute	Data read
1	35.20965ms	89.986µs	33.358924ms	54.546µs	1.760667ms	757ns
2	35.803691ms	99.862µs	33.860305ms	76.742µs	3.311725ms	1.987µs

Table 3.1: Comparison between single or parallel pipeline in Vulkano

N. Pipelines	Init	Setup	Data alloc/load	Data bind	Compute	Data read
1	36.578921ms	208.078µs	4.702801ms	65.195µs	3.164238ms	4.121µs
2	36.946597ms	250.905µs	5.391748ms	110.799µs	6.280969ms	6.084µs

Table 3.2: Comparison between single or parallel pipeline in Rust-GPU

Difference: documentation

The last difference between the two crates is a more practical one: Vulkano is better documented compared to Rust-GPU. Since the latter relies on multiple back-ends, then it just provides a starting guide, and the functions and objects descriptions must be retrieved in the specific back-end documentation.
To be complete it is necessary to say that Vulkano is not fully documented, but the amount of informations provided is very broad.

(DOMANDA: ho notato che Vulkano è molto più veloce nella build quando si fa la run, ed è molto più veloce quando si fanno i benchmark.Quindi aggiungerei una cosa di questo tipo. Va bene o è troppo corta?) As Vulkano has lower dependencies, we experienced that it is significantly faster when it comes to building the project

4 | Figures, Tables and Algorithms

4.1. Algorithms

Listing for Vulkano allocation of the input vector

Listing 4.1: Input vector allocation

```
let input_array = Buffer::from_iter(
        &memory_allocator,
        BufferCreateInfo {
                usage: BufferUsage::STORAGE_BUFFER,
                ..Default::default()
        },
        AllocationCreateInfo {
                usage: MemoryUsage::Download,
                ..Default::default()
        },
        data_iterator,
)
.unwrap();
```

Listing for Rust-GPU allocation of the input vector

Listing 4.2: Input vector allocation

```
// actual_input_vector is Vec<u32>, for instance

let input_vector_converted: &[u8] =
        bytemuck::cast_slice(actual_input_vector);

let input_buffer = device.create_buffer_init(
        &wgpu::util::BufferInitDescriptor {
                label: Some("Input␣array"),
```

```
                contents: input_vector_converted,
                usage: wgpu::BufferUsages::STORAGE
                        | wgpu::BufferUsages::COPY_DST
                        | wgpu::BufferUsages::COPY_SRC,
        }
);
```

(Occupa troppo spazio, cercherò il modo di ridurlo) Shader definition for Vulkano

Listing 4.3: FlatMap shader for Vulkano (flatMap.glsl)

```
#version 450

layout(local_size_x = 16, local_size_y = 1, local_size_z = 1) in;

layout(set = 0, binding = 0) buffer ResultArray {
        uint input_array[];
};

layout(set = 0, binding = 0) buffer ResultArray {
        uint output_array[];
};

void main() {
        uint idx_to_read = gl_GlobalInvocationID.x +
                gl_WorkGroupID.y * gl_WorkGroupSize.x *
                        * gl_NumWorkGroups.x +
                gl_WorkGroupID.z * gl_WorkGroupSize.x *
                        * gl_NumWorkGroups.x * gl_NumWorkGroups.y;
        uint data_to_write = input_array[idx_to_read] * input_array[idx_to_read];
        uint idx_to_write = idx_to_read * 2;

        output_array[idx_to_write] =  data_to_write;
        output_array[idx_to_write + 1] =  data_to_write;
}
```

Shader definition for Rust-GPU

Listing 4.4: FlatMap shader for Rust-GPU (flatMap.rs)

```
#[spirv(compute(threads(256)))]
pub fn main(
#[spirv(global_invocation_id)] global_invocation_id: UVec3,
#[spirv(workgroup_id)] workgroup_id: UVec3,
#[spirv(num_workgroups)] num_workgroups: UVec3,
#[spirv(storage_buffer, descriptor_set = 0, binding = 0)]
        input: &mut[u32],
#[spirv(storage_buffer, descriptor_set = 0, binding = 1)]
        output: &mut[u32],
) {
        let workgroup_size: UVec3 = UVec3 { x: 256, y: 1, z: 1 };

        let idx_to_read = (global_invocation_id.x +
                        workgroup_id.y * workgroup_size.x *
                                * num_workgroups.x +
                        workgroup_id.z * workgroup_size.x *
                                * num_workgroups.x * num_workgroups.y
                ) as usize;

        let to_write_data = input[idx_to_read] * input[idx_to_read];

        output[idx_to_read * 2] = to_write_data;
        output[idx_to_read * 2 + 1] = to_write_data;
}
```

Listing for Vulkano allocation of the output vector
(if FlatMap produces *output_duplication_times* times the single output)

Listing 4.5: Output vector allocation

```
let output_array = Buffer::from_iter(
        &memory_allocator,
        BufferCreateInfo {
                usage: BufferUsage::STORAGE_BUFFER,
                ..Default::default()
        },
        AllocationCreateInfo {
                usage: MemoryUsage::Download,
                ..Default::default()
```

```
        },
        0..(data_size * output_duplication_times),
)
.unwrap();
```

Listing for Rust-GPU allocation of the output vector
(if FlatMap produces *output_duplication_times* times the single output)

Listing 4.6: Output vector allocation

```
/* the CPU copies here the result of the
        GPU operations */
let readback_buffer = device.create_buffer(
        &wgpu::BufferDescriptor {
                label: None,
                size: (conversion_length * 2) as wgpu::BufferAddress,
                usage: wgpu::BufferUsages::MAP_READ |
                        wgpu::BufferUsages::COPY_DST,
                mapped_at_creation: false,
        }
);

let output_buffer = device.create_buffer(
        &wgpu::BufferDescriptor {
                label: None,
                size: (conversion_length * 2) as wgpu::BufferAddress,
                usage: wgpu::BufferUsages::STORAGE
                        | wgpu::BufferUsages::COPY_DST
                        | wgpu::BufferUsages::COPY_SRC,
                mapped_at_creation: false,
        }
);
```

Here we show how to run a series of two pipelines on Rust-GPU (some details are omitted)

Listing 4.7: Rust-GPU series of pipelines

```
let mut encoder: CommandEncoder = device.create_command_encoder(
        &wgpu::CommandEncoderDescriptor { label: None });

```

```
{
        let mut cpass = encoder.begin_compute_pass(
                &wgpu::ComputePassDescriptor { label: None }
        );
        cpass.set_bind_group(0, &bind_group, &[]);
        cpass.set_pipeline(&compute_pipeline1);
        cpass.dispatch_workgroups(num_workgroups);
        cpass.dispatch_workgroups(1, 1, 1);
}
let mut encoders :Vec<CommandBuffer> = Vec::new();
encoders.push(encoder.finish());
queue.submit(encoders);
```

Here we show how to run in parallel two pipelines on Rust-GPU (some details are omitted)

Listing 4.8: Rust-GPU parallel of pipelines

```
let mut encoder1: CommandEncoder = device.create_command_encoder(
        &wgpu::CommandEncoderDescriptor { label: None });

let mut encoder2: CommandEncoder = device.create_command_encoder(
        &wgpu::CommandEncoderDescriptor { label: None });



{
        let mut cpass1 = encoder1.begin_compute_pass(
                &wgpu::ComputePassDescriptor { label: None }
        );
        let mut cpass2 = encoder2.begin_compute_pass(
                &wgpu::ComputePassDescriptor { label: None }
        );
        cpass1.set_bind_group(0, &bind_group1, &[]);
        cpass1.set_pipeline(&compute_pipeline1);
        cpass2.set_bind_group(0, &bind_group2, &[]);
        cpass2.set_pipeline(&compute_pipeline2);

        cpass1.dispatch_workgroups(num_workgroups1);
        cpass2.dispatch_workgroups(1, 1, 1);
```

```
}

let mut encoders: Vec<CommandBuffer> = Vec::new();
encoders.push(encoder1.finish());
encoders.push(encoder2.finish());

queue.submit(encoders);
```

Here we show how to run a series of two pipelines on Vulkano (some details are omitted)

Listing 4.9: Vulkano series of pipelines

```
let mut builder = AutoCommandBufferBuilder::primary(
        &command_buffer_allocator,
        queue.queue_family_index(),
        CommandBufferUsage::SimultaneousUse,
)
.unwrap();

builder
        .bind_pipeline_compute(first_pipeline.clone())
        .bind_descriptor_sets(
                PipelineBindPoint::Compute,
                first_pipeline.layout().clone(),
                0,
                single_set,
        );

builder.
        .dispatch(num_workgroups)
        .unwrap()
        .bind_pipeline_compute(second_pipeline.clone())
        .dispatch([1,1,1])
        .unwrap();

let command_buffer = builder.build().unwrap();

let future = sync::now(device.clone())
        .then_execute(queue, command_buffer)
```

```
        .unwrap()
        .then_signal_fence_and_flush()
        .unwrap()
        ;

future.wait(None).unwrap();
```

Here we show how to run in parallel two pipelines on Vulkano (some details are omitted)

Listing 4.10: Vulkano parallel of pipelines

```
 let mut builder1 = AutoCommandBufferBuilder::primary(
        &command_buffer_allocator,
        queue.queue_family_index(),
        CommandBufferUsage::OneTimeSubmit,
        )
        .unwrap();

let mut builder2 = AutoCommandBufferBuilder::primary(
        &command_buffer_allocator,
        queue.queue_family_index(),
        CommandBufferUsage::OneTimeSubmit,
        )
        .unwrap();


builder1
        .bind_pipeline_compute(pipeline1.clone())
        .bind_descriptor_sets(
                PipelineBindPoint::Compute,
                pipeline1.layout().clone(),
                0,
                single_set1.clone(),
        );

builder2
        .bind_pipeline_compute(pipeline2.clone())
        .bind_descriptor_sets(
                PipelineBindPoint::Compute,
```

```
                pipeline2.layout().clone(),
                0,
                single_set2.clone(),
        );

builder1
        .dispatch(num_workgroups)
        .unwrap();

builder2
        .dispatch([1, 1, 1])
        .unwrap();




let command_buffer1 = builder1.build().unwrap();
let command_buffer2 = builder2.build().unwrap();

let future1 = sync::now(device.clone())
        .then_execute(queue.clone(), command_buffer1)
        .unwrap()
        .then_signal_fence_and_flush()
        .unwrap();

let future2 = sync::now(device)
        .then_execute(queue, command_buffer2)
        .unwrap()
        .then_signal_fence_and_flush()
        .unwrap();

let handle1 = tokio::spawn(future1);
let handle2 = tokio::spawn(future2);

join!(handle1, handle2);
```

5 | Conclusions and future developments

Our goal was to explore all the available possibilities that would allow us to run code on the GPU using Rust, in order to speed up the computations needed by Noir. First of all, this thesis shows how much interest there is in the Rust community for a crate that allows to run code on GPU, given the number of available attempts. Unfortunately, the best working ones focus on graphics APIs, considering the general computing aspect of the GPU as a side goal. This makes us pay some initialization cost for objects that would not be required if a proper CUDA-focused crate would have been available. But regardless of these problems, we can say that using GPUs proves to be a good choice for this task: the amount of time saved to perform operations is remarkable, with both crates.

Given what shown here, Vulkano is faster in all the stages, even if not with all the input sizes. But when looking for the best crate, we should focus on the case where we will find ourselves in the most, which is when there is a big amount of data, and there Vulkano is preferable. We cannot say the same when it comes to flexibility, as it does not allow to run pipelines in parallel.

We already addressed the difference between the two crates regarding the shading language, and there both present some advantages, so it is not easy to say which one is preferable.

Rust-GPU, on the other hand, is better when it comes to pipeline handling, as it can run pipelines in parallel, while Vulkano does not allow it.

Since both present fundamental advantages that the other does not, neither can really be considered the definitive solution, so it is probably better to continue checking the shown crates and to look for new ones that might become more prominent in the future, that solve all the reported problems.

After choosing the most suitable crate, it will be possible to start its integration with Noir, and to test it with some simple operator, like FlatMap, moving then to parallelizing both operators and data, which means running more operator at the same time and computing each operator function in parallel on data.

Bibliography

[1] Arrayfire. Arrayfire crate, 2023. URL https://arrayfire.org/arrayfire-rust/arrayfire/index.html.

[2] Arrayfire. Unified arrayfire backend, 2023. URL https://arrayfire.org/docs/unifiedbackend.htm#gsc.tab=0.

[3] Ash-rs. Ash crate, 2023. URL https://github.com/ash-rs/ash.

[4] K. Barker. Opencl3 crate, 2023. URL https://github.com/kenba/opencl3.

[5] M. Donadoni and E. Morassutto. Noir: Design, implementation and evaluation of a streaming and batch processing framework. Master's thesis, Politecnico di Milano, 2021.

[6] F. Duarte. Amount of data created daily, 2023. URL https://explodingtopics.com/blog/data-generated-per-day.

[7] Embark-studios. Rust-gpu crate, 2023. URL https://github.com/EmbarkStudios/rust-gpu.

[8] Filecoin. Rust-gpu-tools crate, 2023. URL https://github.com/filecoin-project/rust-gpu-tools.

[9] Google. Shaderc-rs crate, 2023. URL https://github.com/google/shaderc-rs.

[10] E. Holk, M. Pathirage, A. Chauhan, A. Lumsdaine, and N. D. Matsakis. Gpu programming in rust: Implementing high-level abstractions in a systems-level language. In *2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum*, pages 315–324, 2013. doi: 10.1109/IPDPSW.2013.173.

[11] M. Khalilov and A. Timoveev. Performance analysis of cuda, openacc and openmp programming models on tesla v100 gpu.

[12] L. D. Martini. Micrometer crate, 2023. URL https://docs.rs/micrometer/latest/micrometer/.

[13] Rust-GPU. Rust-cuda crate, 2023. URL https://github.com/Rust-GPU/Rust-CUDA.

[14] Rustacuda. Rustacuda crate, 2018. URL https://docs.rs/crate/rustacuda/latest.

[15] Vulkano-rs. Vulkano-rs crate, 2023. URL https://github.com/vulkano-rs/vulkano.git.

[16] C. Winston. Emu crate, 2021. URL https://github.com/calebwin/emu.